jp-morgan-2

April 1, 2026

[24]:
```
##Task 1: Data Cleaning and Formatting

import pandas as pd
df=pd.read_excel(r"c:\Users\Admin\Downloads\jp_morgan (1).xlsx")
print (df)
```

```
     TransactionID CustomerID AccountID AccountType TransactionType  \
0                3   CUST2412  ACC80131        Loan      Withdrawal
1               32   CUST1467  ACC74631     Current      Withdrawal
2                9   CUST2699  ACC39482        Loan      Withdrawal
3               42   CUST9535  ACC82947     Current      Withdrawal
4              166   CUST7459  ACC39500      Credit         Payment
..             …        …       …          …              …
795            157   CUST4346  ACC48303     Savings         Payment
796             46   CUST7120  ACC42710     Current        Transfer
797             79   CUST5574  ACC89098     Current      Withdrawal
798            189   CUST9754  ACC86784        Loan         Deposit
799            175   CUST3725  ACC34119        Loan      Withdrawal

              Product    Firm Region    Manager      TransactionDate  \
0      Personal Loan  Firm C   West  Manager 3  2023-06-08 00:00:00
1          Home Loan  Firm D  North  Manager 2  2023-08-11 00:00:00
2        Credit Card  Firm D   West  Manager 4           15-05-2024
3          Home Loan  Firm A  South  Manager 4           30-04-2023
4          Home Loan  Firm D  South  Manager 4           16-02-2023
..               …     …    …        …                  …
795      Mutual Fund  Firm E  North  Manager 3  2023-09-04 00:00:00
796      Mutual Fund  Firm B  North  Manager 4           22-03-2024
797  Savings Account  Firm D  North  Manager 1           22-10-2023
798      Mutual Fund  Firm B   West  Manager 4           28-11-2023
799  Savings Account  Firm D  South  Manager 2           23-04-2023

     TransactionAmount  AccountBalance  RiskScore  CreditRating  TenureMonths
0          33759.690570    126486.40830   0.225824           611            89
1          69319.199330     24834.76291   0.335717           817           174
2          42831.484830    123007.43530   0.572453           332            31
3          70903.796970     73073.64225   0.571993           626            92
4          21948.973550    113405.32820   0.380675           411            13
```

```
..                 …             …         …            …            …
795      105645.967500     13571.39382   0.507578          815            8
796      113961.373300     90617.07646  -0.037634          338          203
797       -6892.749584     71282.75779   0.143038          818          200
798       45433.510640    103798.75240   0.245964          528          239
799       76045.706210     60390.12224   0.582511          331           89

[800 rows x 15 columns]
```

[31]:
```
##Remove/treat any special characters or non-numeric entries from financial␣
 ↪fields.

print(df[['TransactionAmount','AccountBalance']].head())
print (df)
```

```
   TransactionAmount  AccountBalance
0        33759.69057    126486.40830
1        69319.19933     24834.76291
2        42831.48483    123007.43530
3        70903.79697     73073.64225
4        21948.97355    113405.32820
     TransactionID CustomerID AccountID AccountType TransactionType  \
0                3   CUST2412  ACC80131        Loan      Withdrawal
1               32   CUST1467  ACC74631     Current      Withdrawal
2                9   CUST2699  ACC39482        Loan      Withdrawal
3               42   CUST9535  ACC82947     Current      Withdrawal
4              166   CUST7459  ACC39500      Credit         Payment
..             …        …         …          …              …
795            157   CUST4346  ACC48303     Savings         Payment
796             46   CUST7120  ACC42710     Current        Transfer
797             79   CUST5574  ACC89098     Current      Withdrawal
798            189   CUST9754  ACC86784        Loan         Deposit
799            175   CUST3725  ACC34119        Loan      Withdrawal

             Product    Firm Region    Manager TransactionDate  \
0      Personal Loan  Firm C   West  Manager 3      2023-06-08
1          Home Loan  Firm D  North  Manager 2      2023-08-11
2        Credit Card  Firm D   West  Manager 4      2024-05-15
3          Home Loan  Firm A  South  Manager 4      2023-04-30
4          Home Loan  Firm D  South  Manager 4      2023-02-16
..               …     …    …         …              …
795      Mutual Fund  Firm E  North  Manager 3      2023-09-04
796      Mutual Fund  Firm B  North  Manager 4      2024-03-22
797  Savings Account  Firm D  North  Manager 1      2023-10-22
798      Mutual Fund  Firm B   West  Manager 4      2023-11-28
799  Savings Account  Firm D  South  Manager 2      2023-04-23

     TransactionAmount  AccountBalance  RiskScore  CreditRating  TenureMonths
```

```
0          33759.690570    126486.40830   0.225824          611           89
1          69319.199330     24834.76291   0.335717          817          174
2          42831.484830    123007.43530   0.572453          332           31
3          70903.796970     73073.64225   0.571993          626           92
4          21948.973550    113405.32820   0.380675          411           13
..                  …             …          …             …            …
795       105645.967500     13571.39382   0.507578          815            8
796       113961.373300     90617.07646  -0.037634          338          203
797        -6892.749584     71282.75779   0.143038          818          200
798        45433.510640    103798.75240   0.245964          528          239
799        76045.706210     60390.12224   0.582511          331           89

[800 rows x 15 columns]
```

[25]:
```
df.dtypes
```

[25]:
```
TransactionID          int64
CustomerID            object
AccountID             object
AccountType           object
TransactionType       object
Product               object
Firm                  object
Region                object
Manager               object
TransactionDate       object
TransactionAmount    float64
AccountBalance       float64
RiskScore            float64
CreditRating           int64
TenureMonths           int64
dtype: object
```

[27]:
```
import pandas as pd

# Clean financial columns
df['TransactionAmount'] = pd.to_numeric(df['TransactionAmount'].astype(str).str.
  ↪replace(r'[^0-9.-]', '', regex=True), errors='coerce')
print (df)
```

```
     TransactionID CustomerID AccountID AccountType TransactionType  \
0                3   CUST2412  ACC80131        Loan      Withdrawal
1               32   CUST1467  ACC74631     Current      Withdrawal
2                9   CUST2699  ACC39482        Loan      Withdrawal
3               42   CUST9535  ACC82947     Current      Withdrawal
4              166   CUST7459  ACC39500      Credit         Payment
..             …        …         …          …              …
795            157   CUST4346  ACC48303     Savings         Payment
```

```
796              46   CUST7120  ACC42710     Current        Transfer
797              79   CUST5574  ACC89098     Current      Withdrawal
798             189   CUST9754  ACC86784        Loan         Deposit
799             175   CUST3725  ACC34119        Loan      Withdrawal

              Product    Firm Region    Manager      TransactionDate  \
0      Personal Loan  Firm C   West  Manager 3  2023-06-08 00:00:00
1          Home Loan  Firm D  North  Manager 2  2023-08-11 00:00:00
2        Credit Card  Firm D   West  Manager 4           15-05-2024
3          Home Loan  Firm A  South  Manager 4           30-04-2023
4          Home Loan  Firm D  South  Manager 4           16-02-2023
..               …     …    …        …                   …
795      Mutual Fund  Firm E  North  Manager 3  2023-09-04 00:00:00
796      Mutual Fund  Firm B  North  Manager 4           22-03-2024
797  Savings Account  Firm D  North  Manager 1           22-10-2023
798      Mutual Fund  Firm B   West  Manager 4           28-11-2023
799  Savings Account  Firm D  South  Manager 2           23-04-2023

     TransactionAmount  AccountBalance  RiskScore  CreditRating  TenureMonths
0          33759.690570    126486.40830   0.225824           611            89
1          69319.199330     24834.76291   0.335717           817           174
2          42831.484830    123007.43530   0.572453           332            31
3          70903.796970     73073.64225   0.571993           626            92
4          21948.973550    113405.32820   0.380675           411            13
..                  …              …          …           …           …
795       105645.967500     13571.39382   0.507578           815             8
796       113961.373300     90617.07646  -0.037634           338           203
797        -6892.749584     71282.75779   0.143038           818           200
798        45433.510640    103798.75240   0.245964           528           239
799        76045.706210     60390.12224   0.582511           331            89

[800 rows x 15 columns]
```

[28]:
```
## Convert currency amounts into numerical format

df['AccountBalance'] = pd.to_numeric(df['AccountBalance'].astype(str).str.
 ↪replace(r'[^0-9.-]', '', regex=True), errors='coerce')
print (df)
```

```
     TransactionID CustomerID AccountID AccountType TransactionType  \
0                3   CUST2412  ACC80131        Loan      Withdrawal
1               32   CUST1467  ACC74631     Current      Withdrawal
2                9   CUST2699  ACC39482        Loan      Withdrawal
3               42   CUST9535  ACC82947     Current      Withdrawal
4              166   CUST7459  ACC39500      Credit         Payment
..             …        …       …          …               …
795            157   CUST4346  ACC48303     Savings         Payment
796             46   CUST7120  ACC42710     Current        Transfer
```

```
797             79   CUST5574  ACC89098     Current      Withdrawal
798            189   CUST9754  ACC86784        Loan         Deposit
799            175   CUST3725  ACC34119        Loan      Withdrawal

             Product    Firm Region    Manager      TransactionDate  \
0      Personal Loan  Firm C   West  Manager 3  2023-06-08 00:00:00
1          Home Loan  Firm D  North  Manager 2  2023-08-11 00:00:00
2        Credit Card  Firm D   West  Manager 4           15-05-2024
3          Home Loan  Firm A  South  Manager 4           30-04-2023
4          Home Loan  Firm D  South  Manager 4           16-02-2023
..               …     …    …        …                  …
795      Mutual Fund  Firm E  North  Manager 3  2023-09-04 00:00:00
796      Mutual Fund  Firm B  North  Manager 4           22-03-2024
797  Savings Account  Firm D  North  Manager 1           22-10-2023
798      Mutual Fund  Firm B   West  Manager 4           28-11-2023
799  Savings Account  Firm D  South  Manager 2           23-04-2023

     TransactionAmount  AccountBalance  RiskScore  CreditRating  TenureMonths
0          33759.690570    126486.40830   0.225824           611            89
1          69319.199330     24834.76291   0.335717           817           174
2          42831.484830    123007.43530   0.572453           332            31
3          70903.796970     73073.64225   0.571993           626            92
4          21948.973550    113405.32820   0.380675           411            13
..                  …             …         …            …           …
795       105645.967500     13571.39382   0.507578           815             8
796       113961.373300     90617.07646  -0.037634           338           203
797        -6892.749584     71282.75779   0.143038           818           200
798        45433.510640    103798.75240   0.245964           528           239
799        76045.706210     60390.12224   0.582511           331            89

[800 rows x 15 columns]
```

[29]:
```
df.isnull().sum()
```

[29]:
```
TransactionID        0
CustomerID           0
AccountID            0
AccountType          0
TransactionType      0
Product              0
Firm                 0
Region               0
Manager              0
TransactionDate      0
TransactionAmount    0
AccountBalance       0
RiskScore            0
```

```
CreditRating         0
TenureMonths         0
dtype: int64
```

[30]:
```
##Validate and format date columns.

df['TransactionDate'] = pd.to_datetime(df['TransactionDate'], errors='coerce')
print (df)
```

```
     TransactionID CustomerID AccountID AccountType TransactionType  \
0                3   CUST2412  ACC80131        Loan      Withdrawal
1               32   CUST1467  ACC74631     Current      Withdrawal
2                9   CUST2699  ACC39482        Loan      Withdrawal
3               42   CUST9535  ACC82947     Current      Withdrawal
4              166   CUST7459  ACC39500      Credit         Payment
..             …        …        …          …              …
795            157   CUST4346  ACC48303     Savings         Payment
796             46   CUST7120  ACC42710     Current        Transfer
797             79   CUST5574  ACC89098     Current      Withdrawal
798            189   CUST9754  ACC86784        Loan         Deposit
799            175   CUST3725  ACC34119        Loan      Withdrawal

              Product    Firm Region    Manager TransactionDate  \
0      Personal Loan  Firm C   West  Manager 3      2023-06-08
1          Home Loan  Firm D  North  Manager 2      2023-08-11
2        Credit Card  Firm D   West  Manager 4      2024-05-15
3          Home Loan  Firm A  South  Manager 4      2023-04-30
4          Home Loan  Firm D  South  Manager 4      2023-02-16
..               …     …    …        …              …
795      Mutual Fund  Firm E  North  Manager 3      2023-09-04
796      Mutual Fund  Firm B  North  Manager 4      2024-03-22
797  Savings Account  Firm D  North  Manager 1      2023-10-22
798      Mutual Fund  Firm B   West  Manager 4      2023-11-28
799  Savings Account  Firm D  South  Manager 2      2023-04-23

     TransactionAmount  AccountBalance  RiskScore  CreditRating  TenureMonths
0          33759.690570    126486.40830   0.225824           611            89
1          69319.199330     24834.76291   0.335717           817           174
2          42831.484830    123007.43530   0.572453           332            31
3          70903.796970     73073.64225   0.571993           626            92
4          21948.973550    113405.32820   0.380675           411            13
..                  …             …         …            …           …
795       105645.967500     13571.39382   0.507578           815             8
796       113961.373300     90617.07646  -0.037634           338           203
797        -6892.749584     71282.75779   0.143038           818           200
798        45433.510640    103798.75240   0.245964           528           239
799        76045.706210     60390.12224   0.582511           331            89
```

[800 rows x 15 columns]

[32]:
```
##Ensure account types and transaction categories are standardized.

df['AccountType'] = df['AccountType'].str.strip()     ##taskt 1 -4
df['TransactionType'] = df['TransactionType'].str.strip()
print (df)
```

```
     TransactionID CustomerID AccountID AccountType TransactionType  \
0                3   CUST2412  ACC80131        Loan      Withdrawal
1               32   CUST1467  ACC74631     Current      Withdrawal
2                9   CUST2699  ACC39482        Loan      Withdrawal
3               42   CUST9535  ACC82947     Current      Withdrawal
4              166   CUST7459  ACC39500      Credit         Payment
..             …        …        …          …              …
795            157   CUST4346  ACC48303     Savings         Payment
796             46   CUST7120  ACC42710     Current        Transfer
797             79   CUST5574  ACC89098     Current      Withdrawal
798            189   CUST9754  ACC86784        Loan         Deposit
799            175   CUST3725  ACC34119        Loan      Withdrawal

              Product    Firm Region    Manager TransactionDate  \
0      Personal Loan  Firm C   West  Manager 3      2023-06-08
1          Home Loan  Firm D  North  Manager 2      2023-08-11
2        Credit Card  Firm D   West  Manager 4      2024-05-15
3          Home Loan  Firm A  South  Manager 4      2023-04-30
4          Home Loan  Firm D  South  Manager 4      2023-02-16
..               …     …    …        …              …
795      Mutual Fund  Firm E  North  Manager 3      2023-09-04
796      Mutual Fund  Firm B  North  Manager 4      2024-03-22
797  Savings Account  Firm D  North  Manager 1      2023-10-22
798      Mutual Fund  Firm B   West  Manager 4      2023-11-28
799  Savings Account  Firm D  South  Manager 2      2023-04-23

     TransactionAmount  AccountBalance  RiskScore  CreditRating  TenureMonths
0          33759.690570    126486.40830   0.225824           611            89
1          69319.199330     24834.76291   0.335717           817           174
2          42831.484830    123007.43530   0.572453           332            31
3          70903.796970     73073.64225   0.571993           626            92
4          21948.973550    113405.32820   0.380675           411            13
..                  …             …         …            …           …
795       105645.967500     13571.39382   0.507578           815             8
796       113961.373300     90617.07646  -0.037634           338           203
797        -6892.749584     71282.75779   0.143038           818           200
798        45433.510640    103798.75240   0.245964           528           239
799        76045.706210     60390.12224   0.582511           331            89
```

```
[800 rows x 15 columns]
```

[33]:
```
df['AccountType'] = df['AccountType'].str.title()    ##taskt 1 -4
df['TransactionType'] = df['TransactionType'].str.title()
print (df)
```

```
     TransactionID CustomerID AccountID AccountType TransactionType  \
0                3   CUST2412  ACC80131        Loan      Withdrawal
1               32   CUST1467  ACC74631     Current      Withdrawal
2                9   CUST2699  ACC39482        Loan      Withdrawal
3               42   CUST9535  ACC82947     Current      Withdrawal
4              166   CUST7459  ACC39500      Credit         Payment
..             …        …        …          …              …
795            157   CUST4346  ACC48303     Savings         Payment
796             46   CUST7120  ACC42710     Current        Transfer
797             79   CUST5574  ACC89098     Current      Withdrawal
798            189   CUST9754  ACC86784        Loan         Deposit
799            175   CUST3725  ACC34119        Loan      Withdrawal

              Product    Firm Region    Manager TransactionDate  \
0      Personal Loan  Firm C   West  Manager 3      2023-06-08
1          Home Loan  Firm D  North  Manager 2      2023-08-11
2        Credit Card  Firm D   West  Manager 4      2024-05-15
3          Home Loan  Firm A  South  Manager 4      2023-04-30
4          Home Loan  Firm D  South  Manager 4      2023-02-16
..               …     …    …        …              …
795      Mutual Fund  Firm E  North  Manager 3      2023-09-04
796      Mutual Fund  Firm B  North  Manager 4      2024-03-22
797  Savings Account  Firm D  North  Manager 1      2023-10-22
798      Mutual Fund  Firm B   West  Manager 4      2023-11-28
799  Savings Account  Firm D  South  Manager 2      2023-04-23

     TransactionAmount  AccountBalance  RiskScore  CreditRating  TenureMonths
0          33759.690570    126486.40830   0.225824           611            89
1          69319.199330     24834.76291   0.335717           817           174
2          42831.484830    123007.43530   0.572453           332            31
3          70903.796970     73073.64225   0.571993           626            92
4          21948.973550    113405.32820   0.380675           411            13
..                  …             …         …            …           …
795       105645.967500     13571.39382   0.507578           815             8
796       113961.373300     90617.07646  -0.037634           338           203
797        -6892.749584     71282.75779   0.143038           818           200
798        45433.510640    103798.75240   0.245964           528           239
799        76045.706210     60390.12224   0.582511           331            89

[800 rows x 15 columns]
```

[58]:
```
##Task 2: Descriptive Transactional Analysis
## Calculate monthly and yearly summaries of total credits, debits, and net␣
 ↪transaction
#volume.

import pandas as pd

df['TransactionDate'] = pd.to_datetime(df['TransactionDate'])

df['Year'] = df['TransactionDate'].dt.year
df['Month'] = df['TransactionDate'].dt.month

df['Credit'] = df['TransactionAmount'].where(df['TransactionType']=='Credit',0)
df['Debit'] = df['TransactionAmount'].where(df['TransactionType']=='Debit',0)

monthly_summary = df.groupby(['Year','Month'])[['Credit','Debit']].sum().
 ↪reset_index()
monthly_summary['NetVolume'] = monthly_summary['Credit'] -␣
 ↪monthly_summary['Debit']

yearly_summary = df.groupby('Year')[['Credit','Debit']].sum().reset_index()
yearly_summary['NetVolume'] = yearly_summary['Credit'] - yearly_summary['Debit']

print(monthly_summary)
print(yearly_summary)
```

	Year	Month	Credit	Debit	NetVolume
0	2023	1	0.0	0.0	0.0
1	2023	2	0.0	0.0	0.0
2	2023	3	0.0	0.0	0.0
3	2023	4	0.0	0.0	0.0
4	2023	5	0.0	0.0	0.0
5	2023	6	0.0	0.0	0.0
6	2023	7	0.0	0.0	0.0
7	2023	8	0.0	0.0	0.0
8	2023	9	0.0	0.0	0.0
9	2023	10	0.0	0.0	0.0
10	2023	11	0.0	0.0	0.0
11	2023	12	0.0	0.0	0.0
12	2024	1	0.0	0.0	0.0
13	2024	2	0.0	0.0	0.0
14	2024	3	0.0	0.0	0.0
15	2024	4	0.0	0.0	0.0
16	2024	5	0.0	0.0	0.0
17	2024	6	0.0	0.0	0.0
18	2024	7	0.0	0.0	0.0
19	2024	8	0.0	0.0	0.0

```
20  2024      9     0.0    0.0         0.0
21  2024     11     0.0    0.0         0.0
22  2024     12     0.0    0.0         0.0
   Year  Credit  Debit  NetVolume
0  2023     0.0    0.0        0.0
1  2024     0.0    0.0        0.0
```

[53]:
```
##Task 2: Descriptive Transactional Analysis

##Plot trends in total credits vs. debits over time.

df['Credit'] = df['TransactionAmount'].where(df['TransactionType'] ==␣
 ↪'Deposit', 0)

df['Debit'] = df['TransactionAmount'].where(
    df['TransactionType'].isin(['Withdrawal','Payment']), 0)

print (df)
```

```
     TransactionID CustomerID AccountID AccountType TransactionType  \
0                3   CUST2412  ACC80131        Loan      Withdrawal
1               32   CUST1467  ACC74631     Current      Withdrawal
2                9   CUST2699  ACC39482        Loan      Withdrawal
3               42   CUST9535  ACC82947     Current      Withdrawal
4              166   CUST7459  ACC39500      Credit         Payment
..             …        …       …          …              …
795            157   CUST4346  ACC48303     Savings         Payment
796             46   CUST7120  ACC42710     Current        Transfer
797             79   CUST5574  ACC89098     Current      Withdrawal
798            189   CUST9754  ACC86784        Loan         Deposit
799            175   CUST3725  ACC34119        Loan      Withdrawal

             Product    Firm Region    Manager TransactionDate  \
0      Personal Loan  Firm C   West  Manager 3      2023-06-08
1          Home Loan  Firm D  North  Manager 2      2023-08-11
2        Credit Card  Firm D   West  Manager 4      2024-05-15
3          Home Loan  Firm A  South  Manager 4      2023-04-30
4          Home Loan  Firm D  South  Manager 4      2023-02-16
..               …     …    …        …              …
795      Mutual Fund  Firm E  North  Manager 3      2023-09-04
796      Mutual Fund  Firm B  North  Manager 4      2024-03-22
797  Savings Account  Firm D  North  Manager 1      2023-10-22
798      Mutual Fund  Firm B   West  Manager 4      2023-11-28
799  Savings Account  Firm D  South  Manager 2      2023-04-23

     TransactionAmount  AccountBalance  RiskScore  CreditRating  TenureMonths  \
0         33759.690570    126486.40830   0.225824           611            89
1         69319.199330     24834.76291   0.335717           817           174
```

```
2          42831.484830    123007.43530   0.572453          332           31
3          70903.796970     73073.64225   0.571993          626           92
4          21948.973550    113405.32820   0.380675          411           13
..                  …             …          …            …          …
795       105645.967500     13571.39382   0.507578          815            8
796       113961.373300     90617.07646  -0.037634          338          203
797        -6892.749584     71282.75779   0.143038          818          200
798        45433.510640    103798.75240   0.245964          528          239
799        76045.706210     60390.12224   0.582511          331           89

     Year  Month       Credit          Debit YearMonth
0    2023      6      0.00000   33759.690570   2023-06
1    2023      8      0.00000   69319.199330   2023-08
2    2024      5      0.00000   42831.484830   2024-05
3    2023      4      0.00000   70903.796970   2023-04
4    2023      2      0.00000   21948.973550   2023-02
..    …    …           …             …       …
795  2023      9      0.00000  105645.967500   2023-09
796  2024      3      0.00000       0.000000   2024-03
797  2023     10      0.00000   -6892.749584   2023-10
798  2023     11  45433.51064       0.000000   2023-11
799  2023      4      0.00000   76045.706210   2023-04

[800 rows x 20 columns]
```

[54]:
```
##Task 2: Descriptive Transactional Analysis

##Plot trends in total credits vs. debits over time.

df['TransactionDate'] = pd.to_datetime(df['TransactionDate'])
df['YearMonth'] = df['TransactionDate'].dt.to_period('M')
print (df)
```

```
     TransactionID CustomerID AccountID AccountType TransactionType  \
0                3   CUST2412  ACC80131        Loan      Withdrawal
1               32   CUST1467  ACC74631     Current      Withdrawal
2                9   CUST2699  ACC39482        Loan      Withdrawal
3               42   CUST9535  ACC82947     Current      Withdrawal
4              166   CUST7459  ACC39500      Credit         Payment
..             …        …       …          …              …
795            157   CUST4346  ACC48303     Savings         Payment
796             46   CUST7120  ACC42710     Current        Transfer
797             79   CUST5574  ACC89098     Current      Withdrawal
798            189   CUST9754  ACC86784        Loan         Deposit
799            175   CUST3725  ACC34119        Loan      Withdrawal

             Product    Firm Region    Manager TransactionDate  \
0      Personal Loan  Firm C   West  Manager 3      2023-06-08
```

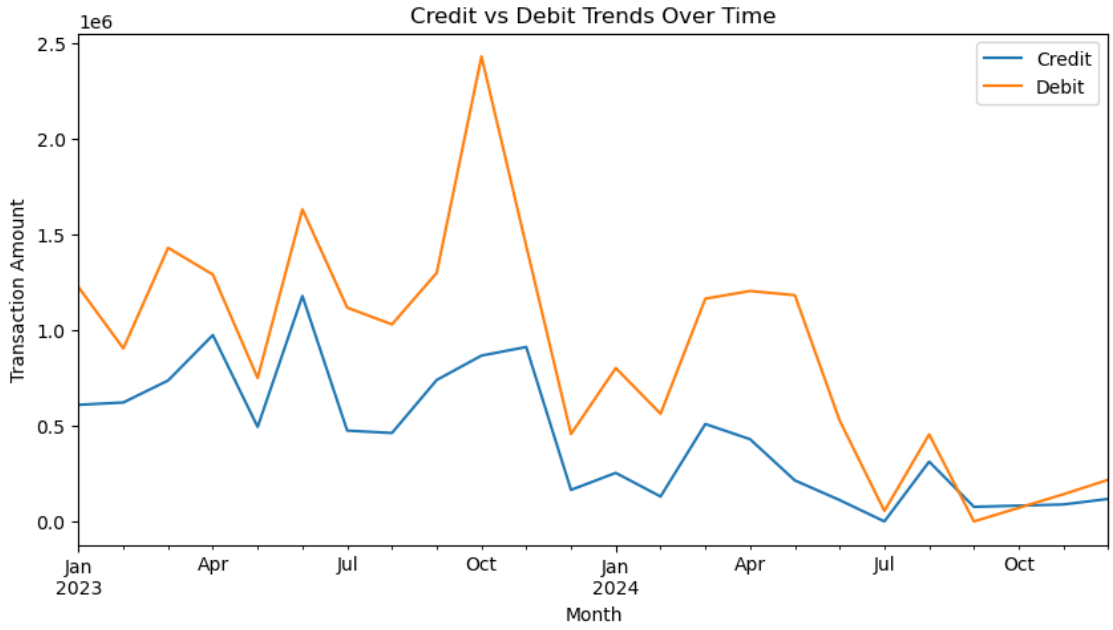
```
1          Home Loan  Firm D  North  Manager 2      2023-08-11
2        Credit Card  Firm D   West  Manager 4      2024-05-15
3          Home Loan  Firm A  South  Manager 4      2023-04-30
4          Home Loan  Firm D  South  Manager 4      2023-02-16
..               …     …    …        …              …
795      Mutual Fund  Firm E  North  Manager 3      2023-09-04
796      Mutual Fund  Firm B  North  Manager 4      2024-03-22
797  Savings Account  Firm D  North  Manager 1      2023-10-22
798      Mutual Fund  Firm B   West  Manager 4      2023-11-28
799  Savings Account  Firm D  South  Manager 2      2023-04-23

     TransactionAmount  AccountBalance  RiskScore  CreditRating  TenureMonths  \
0          33759.690570    126486.40830   0.225824           611            89
1          69319.199330     24834.76291   0.335717           817           174
2          42831.484830    123007.43530   0.572453           332            31
3          70903.796970     73073.64225   0.571993           626            92
4          21948.973550    113405.32820   0.380675           411            13
..                  …              …          …             …             …
795       105645.967500     13571.39382   0.507578           815             8
796       113961.373300     90617.07646  -0.037634           338           203
797        -6892.749584     71282.75779   0.143038           818           200
798        45433.510640    103798.75240   0.245964           528           239
799        76045.706210     60390.12224   0.582511           331            89

     Year  Month       Credit          Debit YearMonth
0    2023      6      0.00000   33759.690570   2023-06
1    2023      8      0.00000   69319.199330   2023-08
2    2024      5      0.00000   42831.484830   2024-05
3    2023      4      0.00000   70903.796970   2023-04
4    2023      2      0.00000   21948.973550   2023-02
..    …    …           …             …       …
795  2023      9      0.00000  105645.967500   2023-09
796  2024      3      0.00000       0.000000   2024-03
797  2023     10      0.00000   -6892.749584   2023-10
798  2023     11  45433.51064       0.000000   2023-11
799  2023      4      0.00000   76045.706210   2023-04

[800 rows x 20 columns]
```

[55]:
```
##Task 2: Descriptive Transactional Analysis

##Plot trends in total credits vs. debits over time.


monthly = df.groupby('YearMonth')[['Credit','Debit']].sum()
print(monthly)
```

```
                Credit         Debit
```

```
YearMonth
2023-01    6.088844e+05  1.223324e+06
2023-02    6.210296e+05  9.032203e+05
2023-03    7.359823e+05  1.429123e+06
2023-04    9.732157e+05  1.289276e+06
2023-05    4.935210e+05  7.492057e+05
2023-06    1.177578e+06  1.629915e+06
2023-07    4.741172e+05  1.116636e+06
2023-08    4.616070e+05  1.028992e+06
2023-09    7.386258e+05  1.298157e+06
2023-10    8.659592e+05  2.428727e+06
2023-11    9.109240e+05  1.438782e+06
2023-12    1.643427e+05  4.564327e+05
2024-01    2.532480e+05  8.009616e+05
2024-02    1.297533e+05  5.622679e+05
2024-03    5.086565e+05  1.163419e+06
2024-04    4.290451e+05  1.203532e+06
2024-05    2.138735e+05  1.181686e+06
2024-06    1.112170e+05  5.270611e+05
2024-07   -4.051526e+02  5.448218e+04
2024-08    3.120289e+05  4.543142e+05
2024-09    7.597006e+04 -4.520526e+02
2024-11    8.828567e+04  1.416748e+05
2024-12    1.172736e+05  2.172183e+05
```

[56]:
```python
import matplotlib.pyplot as plt

monthly.plot(figsize=(10,5))

plt.title("Credit vs Debit Trends Over Time")
plt.xlabel("Month")
plt.ylabel("Transaction Amount")

plt.show()
```



[3]:
```
##Identify top and bottom performing accounts based on net inflow.

import pandas as pd
df = pd.read_excel("C:/Users/Admin/Downloads/jp_morgan (1).xlsx")

df['Credit'] = df['TransactionAmount'].where(df['TransactionType']=='Deposit',0)

df['Debit'] = df['TransactionAmount'].where(
    df['TransactionType'].isin(['Withdrawal','Payment']),0)

account_summary = df.groupby('AccountID')[['Credit','Debit']].sum()

account_summary['NetInflow'] = account_summary['Credit'] -␣
 ↪account_summary['Debit']

top_accounts = account_summary.sort_values(by='NetInflow',ascending=False).
 ↪head(10)

bottom_accounts = account_summary.sort_values(by='NetInflow').head(10)

print("Top Performing Accounts")
print(top_accounts)

print("\nBottom Performing Accounts")
print(bottom_accounts)
```

```
Top Performing Accounts
                Credit          Debit      NetInflow
AccountID
ACC54589   225122.49518       0.000000  225122.495180
ACC67713   196130.57161       0.000000  196130.571610
ACC22036   291395.48314  101812.702048  189582.781092
ACC13357   188200.23339       0.000000  188200.233390
ACC29396   166862.63099       0.000000  166862.630990
ACC74631   253027.51185  102073.407220  150954.104630
ACC87602   162772.56621   11911.998690  150860.567520
ACC92104   147447.29510       0.000000  147447.295100
ACC12182   108490.02780  -16135.998450  124626.026250
ACC89098   116032.57878   -6892.749584  122925.328364

Bottom Performing Accounts
                Credit          Debit      NetInflow
AccountID
ACC18140       0.00000  378278.364630 -378278.364630
ACC99549    2521.84752  337033.401670 -334511.554150
ACC55331       0.00000  329665.651130 -329665.651130
ACC35419       0.00000  296560.677868 -296560.677868
ACC51593       0.00000  288480.103290 -288480.103290
ACC29477   81173.18694  361894.297917 -280721.110977
ACC81631   70661.59074  331563.750460 -260902.159720
ACC11285       0.00000  255253.191210 -255253.191210
ACC58078       0.00000  246152.803390 -246152.803390
ACC61926       0.00000  230588.495848 -230588.495848
```

[4]:
```python
##Identify and flag accounts as dormant or inactive if there is a gap of two␣
 ↪months or more
##between consecutive transactions.

import pandas as pd

df['TransactionDate'] = pd.to_datetime(df['TransactionDate'])

df = df.sort_values(['AccountID','TransactionDate'])

df['PrevDate'] = df.groupby('AccountID')['TransactionDate'].shift(1)

df['GapDays'] = (df['TransactionDate'] - df['PrevDate']).dt.days

df['DormantFlag'] = df['GapDays'] >= 60

dormant_accounts = df[df['DormantFlag']]['AccountID'].unique()

print("Dormant Accounts:")
```

```
print(dormant_accounts)
```

```
Dormant Accounts:
['ACC10996' 'ACC11062' 'ACC11188' 'ACC11285' 'ACC12182' 'ACC12334'
 'ACC13357' 'ACC15228' 'ACC15359' 'ACC15671' 'ACC15925' 'ACC16241'
 'ACC16664' 'ACC17688' 'ACC18057' 'ACC18140' 'ACC19156' 'ACC19178'
 'ACC20297' 'ACC21264' 'ACC21429' 'ACC21719' 'ACC21878' 'ACC22036'
 'ACC22799' 'ACC23736' 'ACC23985' 'ACC24070' 'ACC24508' 'ACC24880'
 'ACC24981' 'ACC25132' 'ACC25811' 'ACC26026' 'ACC26940' 'ACC26956'
 'ACC26973' 'ACC28154' 'ACC28295' 'ACC28305' 'ACC29007' 'ACC29231'
 'ACC29356' 'ACC29396' 'ACC29477' 'ACC29646' 'ACC30787' 'ACC31539'
 'ACC31902' 'ACC32212' 'ACC32627' 'ACC32890' 'ACC33287' 'ACC34119'
 'ACC34431' 'ACC34568' 'ACC34821' 'ACC35163' 'ACC35419' 'ACC36079'
 'ACC38559' 'ACC39161' 'ACC39500' 'ACC39529' 'ACC39544' 'ACC40939'
 'ACC40952' 'ACC42467' 'ACC42710' 'ACC42903' 'ACC43309' 'ACC43771'
 'ACC45101' 'ACC45521' 'ACC45907' 'ACC45951' 'ACC45968' 'ACC46655'
 'ACC47099' 'ACC48303' 'ACC48501' 'ACC49140' 'ACC49395' 'ACC49422'
 'ACC50439' 'ACC50817' 'ACC51009' 'ACC51200' 'ACC51593' 'ACC51971'
 'ACC52131' 'ACC52650' 'ACC53466' 'ACC53865' 'ACC54589' 'ACC55331'
 'ACC55729' 'ACC57516' 'ACC57597' 'ACC57700' 'ACC57872' 'ACC58078'
 'ACC58667' 'ACC60432' 'ACC61827' 'ACC61926' 'ACC62446' 'ACC62809'
 'ACC64022' 'ACC64393' 'ACC64785' 'ACC65144' 'ACC65545' 'ACC66086'
 'ACC66190' 'ACC67701' 'ACC67713' 'ACC69323' 'ACC70314' 'ACC70460'
 'ACC70741' 'ACC71388' 'ACC71426' 'ACC71938' 'ACC72197' 'ACC73104'
 'ACC74656' 'ACC75675' 'ACC75767' 'ACC76699' 'ACC77533' 'ACC77592'
 'ACC77638' 'ACC77773' 'ACC78089' 'ACC78178' 'ACC78581' 'ACC78589'
 'ACC80131' 'ACC80442' 'ACC81631' 'ACC82298' 'ACC82381' 'ACC82926'
 'ACC83005' 'ACC83269' 'ACC83581' 'ACC83848' 'ACC86784' 'ACC88074'
 'ACC88252' 'ACC88286' 'ACC88449' 'ACC88516' 'ACC89098' 'ACC90887'
 'ACC92104' 'ACC92360' 'ACC92558' 'ACC94203' 'ACC94242' 'ACC94907'
 'ACC95164' 'ACC95774' 'ACC96868' 'ACC97225' 'ACC97411' 'ACC99117'
 'ACC99549']
```

[5]:
```
##Task 3: Customer Profile Building
#Group accounts by activity levels: High, Medium, Low based on transaction␣
 ↪frequency on
#your analysis and rubrics. Do not forget to mention the rubric in the headings.
 ↪


import pandas as pd

transaction_counts = df.groupby('AccountID').size().
 ↪reset_index(name='TransactionCount')

def activity_level(count):
    if count > 50:
```

```
        return "High"
    elif count >= 20:
        return "Medium"
    else:
        return "Low"

transaction_counts['ActivityLevel'] = transaction_counts['TransactionCount'].
 ↪apply(activity_level)

print(transaction_counts)
```

```
    AccountID  TransactionCount ActivityLevel
0    ACC10117                 1           Low
1    ACC10996                 4           Low
2    ACC11062                 4           Low
3    ACC11188                 3           Low
4    ACC11285                 6           Low
..        …                 …           …
189  ACC97225                 5           Low
190  ACC97411                 6           Low
191  ACC99117                 6           Low
192  ACC99409                 2           Low
193  ACC99549                 5           Low

[194 rows x 3 columns]
```

[6]:
```
##Segment customers by average balance and transaction volume.

customer_summary = df.groupby('CustomerID').agg({
    'AccountBalance':'mean',
    'TransactionAmount':'sum'
}).reset_index()

customer_summary.rename(columns={
    'AccountBalance':'AvgBalance',
    'TransactionAmount':'TransactionVolume'
}, inplace=True)

def customer_segment(row):
    if row['AvgBalance'] > 100000 and row['TransactionVolume'] > 200000:
        return "Premium"
    elif row['AvgBalance'] > 30000:
        return "Regular"
    else:
        return "Low Value"
```

```
customer_summary['CustomerSegment'] = customer_summary.apply(customer_segment,␣
 ↪axis=1)

print(customer_summary)
```

```
    CustomerID    AvgBalance  TransactionVolume CustomerSegment
0     CUST1042  72029.783760      239707.842520         Regular
1     CUST1114  72200.316843      148429.225518         Regular
2     CUST1121  65967.294726      234165.196290         Regular
3     CUST1189  75396.365689      355664.766380         Regular
4     CUST1223  85890.369912      553100.623290         Regular
..         …           …                 …              …
185   CUST9683  64297.907472      196283.674120         Regular
186   CUST9731  99011.149050       90947.757040         Regular
187   CUST9754  75699.930908      154408.982550         Regular
188   CUST9843  33886.618660      121623.569880         Regular
189   CUST9962  92246.372760      212231.197020         Regular

[190 rows x 4 columns]
```

[10]:
```
#Create profiles for:
# High-net inflow accounts



high_inflow_accounts = account_summary[account_summary['NetInflow'] > 50000]

print("High-Net Inflow Accounts")
print(high_inflow_accounts)
```

```
High-Net Inflow Accounts
    AccountID         Credit          Debit  TransactionCount     AvgBalance  \
1    ACC10996  173932.456600   59601.060630                 4   64046.568590
5    ACC11837   52934.972090       0.000000                 1   97081.482910
6    ACC12182  108490.027800  -16135.998450                 4  111954.461552
8    ACC13357  188200.233390       0.000000                 6   66751.175967
10   ACC15359  118168.271021   63623.517940                 5   84516.019222
16   ACC18057   90849.078650   26690.673580                 2   92368.570175
26   ACC22036  291395.483140  101812.702048                 8   94908.054831
30   ACC23985   87914.119340       0.000000                 3   95572.179197
42   ACC28292   93721.351460       0.000000                 1   37550.014430
49   ACC29396  166862.630990       0.000000                 5   70380.916422
68   ACC37688   89340.899530       0.000000                 3   62688.779050
72   ACC39500  210285.097915   88182.934740                 6   57445.826828
79   ACC42710  216382.793550  108987.579390                 7   76743.856084
89   ACC46953   67572.875620       0.000000                 1   93340.133980
93   ACC49140   80810.263110   -9957.617829                 5   79668.948622
```

100	ACC50817	221018.188930	107460.627380	8	65243.959563
106	ACC52650	54964.303748	0.000000	2	90624.922705
109	ACC54589	225122.495180	0.000000	3	42538.305527
115	ACC57872	118892.676040	0.000000	2	50980.576460
117	ACC58667	179291.583130	83531.572510	5	55285.627332
131	ACC67713	196130.571610	0.000000	5	106158.775192
135	ACC70741	113056.368460	59910.923020	3	72437.560617
140	ACC73104	123955.607800	52354.226000	2	69321.786520
141	ACC74631	253027.511850	102073.407220	6	72988.870535
144	ACC75767	63838.473170	0.000000	4	81803.692923
154	ACC78581	46275.239750	-21431.796840	3	71791.277103
165	ACC83581	152204.186100	65418.351890	5	88684.379424
169	ACC87602	162772.566210	11911.998690	4	78349.172277
173	ACC88449	131865.541830	57645.314650	3	56513.645530
175	ACC89098	116032.578780	-6892.749584	4	63121.423470
179	ACC92104	147447.295100	0.000000	2	123092.693350
183	ACC94242	190591.313130	109271.545750	6	66202.381570
188	ACC96868	72497.702110	0.000000	3	47407.561900
190	ACC97411	159488.631370	84426.869010	6	61783.633875

	NetInflow
1	114331.395970
5	52934.972090
6	124626.026250
8	188200.233390
10	54544.753081
16	64158.405070
26	189582.781092
30	87914.119340
42	93721.351460
49	166862.630990
68	89340.899530
72	122102.163175
79	107395.214160
89	67572.875620
93	90767.880939
100	113557.561550
106	54964.303748
109	225122.495180
115	118892.676040
117	95760.010620
131	196130.571610
135	53145.445440
140	71601.381800
141	150954.104630
144	63838.473170
154	67707.036590
165	86785.834210

```
169  150860.567520
173   74220.227180
175  122925.328364
179  147447.295100
183   81319.767380
188   72497.702110
190   75061.762360
```

[11]:
```
#High-frequency low-balance accounts

high_freq_low_balance = account_summary[
    (account_summary['TransactionCount'] > 50) &
    (account_summary['AvgBalance'] < 10000)
]

print("High-Frequency Low-Balance Accounts")
print(high_freq_low_balance)
```

```
High-Frequency Low-Balance Accounts
Empty DataFrame
Columns: [AccountID, Credit, Debit, TransactionCount, AvgBalance, NetInflow]
Index: []
```

[12]:
```
# Accounts with negative or near-zero balances

negative_balance_accounts = account_summary[
    account_summary['AvgBalance'] <= 0
]

print("Negative or Near-Zero Balance Accounts")
print(negative_balance_accounts)
```

```
Negative or Near-Zero Balance Accounts
Empty DataFrame
Columns: [AccountID, Credit, Debit, TransactionCount, AvgBalance, NetInflow]
Index: []
```

[17]:
```
##Task 4: Financial Risk Identification

##Track accounts with frequent large withdrawals or overdrafts.

large_withdrawals = df[
    (df['TransactionType'] == 'Withdrawal') &
    (df['TransactionAmount'] > 50000)
]
print (large_withdrawals)
```

```
     TransactionID CustomerID AccountID AccountType TransactionType  \
```

```
312            140   CUST9843  ACC10996     Current      Withdrawal
771            155   CUST1747  ACC11285     Savings      Withdrawal
628            195   CUST4679  ACC11285        Loan      Withdrawal
654             76   CUST7887  ACC11285        Loan      Withdrawal
216             69   CUST7120  ACC15228     Savings      Withdrawal
..             …        …         …          …             …
560            154   CUST4780  ACC99117        Loan      Withdrawal
115            122   CUST8344  ACC99117      Credit      Withdrawal
158             50   CUST7395  ACC99409        Loan      Withdrawal
410              3   CUST5609  ACC99549     Savings      Withdrawal
721            111   CUST1189  ACC99549        Loan      Withdrawal

             Product    Firm   Region    Manager TransactionDate  \
312      Credit Card  Firm D     East  Manager 4      2023-01-21
771        Home Loan  Firm C     East  Manager 2      2023-11-20
628    Personal Loan  Firm B    South  Manager 1      2024-05-24
654  Savings Account  Firm B    North  Manager 4      2024-08-06
216    Personal Loan  Firm A    North  Manager 2      2023-07-11
..               …     …      …         …               …
560    Personal Loan  Firm E     East  Manager 3      2023-11-18
115        Home Loan  Firm B     East  Manager 4      2024-01-29
158        Home Loan  Firm A     West  Manager 4      2023-10-12
410    Personal Loan  Firm C     West  Manager 3      2023-11-21
721    Personal Loan  Firm A  Central  Manager 3      2024-06-04

     TransactionAmount  AccountBalance  RiskScore  CreditRating  TenureMonths  \
312        59601.06063     42962.44757   0.483354           363           185
771        58554.13642    162335.77730   0.622845           661           100
628        77545.88069    106139.58270   0.513754           699            75
654        84119.87650     77599.99069  -0.331731           558            17
216        50510.74966     20535.70594   0.528891           315            62
..                 …             …         …             …           …
560        69501.78283     80142.29171   0.490142           484           206
115       129471.46560     73117.85560   0.186813           820           210
158        97158.08018     54295.87766   0.468137           720           135
410        79849.03381    145083.84670   0.429853           593           164
721        68150.18631     60628.36475   0.382551           666           118

     Credit         Debit   PrevDate  GapDays  DormantFlag
312     0.0   59601.06063 2023-01-07     14.0        False
771     0.0   58554.13642 2023-10-24     27.0        False
628     0.0   77545.88069 2024-04-25     29.0        False
654     0.0   84119.87650 2024-06-02     65.0         True
216     0.0   50510.74966 2023-04-14     88.0         True
..      …           …       …      …          …
560     0.0   69501.78283 2023-09-19     60.0         True
115     0.0  129471.46560 2023-11-29     61.0         True
158     0.0   97158.08018        NaT      NaN        False
```

```
410     0.0   79849.03381 2023-03-18    248.0         True
721     0.0   68150.18631 2023-11-25    192.0         True

[112 rows x 20 columns]
```

[20]:
```
withdrawal_counts = large_withdrawals.groupby('AccountID').size().
 ↪reset_index(name='LargeWithdrawalCount')
overdraft_accounts = df[df['AccountBalance'] < 0]
overdraft_counts = overdraft_accounts.groupby('AccountID').size().
 ↪reset_index(name='OverdraftCount')
print (withdrawal_counts)
print (overdraft_accounts)
print (overdraft_counts)
```

```
   AccountID  LargeWithdrawalCount
0   ACC10996                     1
1   ACC11285                     3
2   ACC15228                     2
3   ACC15925                     1
4   ACC16664                     1
..       …                     …
82  ACC95164                     1
83  ACC95774                     1
84  ACC99117                     2
85  ACC99409                     1
86  ACC99549                     2

[87 rows x 2 columns]
     TransactionID CustomerID AccountID AccountType TransactionType  \
568              2   CUST7395  ACC30852     Current        Transfer
756             43   CUST2915  ACC45907        Loan      Withdrawal
147            113   CUST6028  ACC49774      Credit      Withdrawal
121             41   CUST9535  ACC50817     Savings         Deposit
102            168   CUST4769  ACC57516     Current         Payment
621            123   CUST9525  ACC57597     Savings        Transfer
733            102   CUST4373  ACC57700        Loan      Withdrawal
543             31   CUST8461  ACC61926     Savings      Withdrawal
482            180   CUST6947  ACC69323     Current        Transfer
614            119   CUST8318  ACC77638     Current         Deposit

             Product    Firm   Region    Manager TransactionDate  \
568    Personal Loan  Firm D     East  Manager 4      2023-09-12
756  Savings Account  Firm E     East  Manager 3      2023-07-26
147        Home Loan  Firm A    North  Manager 4      2023-11-18
121      Mutual Fund  Firm A     East  Manager 4      2023-10-22
102    Personal Loan  Firm E    South  Manager 3      2023-05-26
621      Credit Card  Firm C  Central  Manager 4      2023-07-26
733    Personal Loan  Firm A     West  Manager 4      2024-04-21
```

```
543      Credit Card  Firm A    South  Manager 2      2023-06-11
482      Mutual Fund  Firm D  Central  Manager 1      2023-11-24
614      Mutual Fund  Firm B  Central  Manager 1      2023-04-30

     TransactionAmount  AccountBalance  RiskScore  CreditRating  TenureMonths  \
568        47335.25073   -13018.400900   0.280604           320             8
756        17331.90707    -3430.668059   0.708868           437           195
147        22161.01926     -584.724017   0.486208           697           159
121        29747.40747   -13812.693060   0.199034           624           167
102        54309.34491    -1308.418354   1.079204           760            81
621        37256.51091   -12919.394960   0.254919           403            69
733        72713.06247    -1164.041546   0.280531           439            82
543        67370.55910    -9976.162775   0.173516           475           170
482        49101.74069    -8873.804269   0.141274           343           229
614        72351.37574    -4782.075751   0.916203           721           233

          Credit        Debit   PrevDate  GapDays  DormantFlag
568      0.00000      0.00000 2023-07-17     57.0        False
756      0.00000  17331.90707 2023-07-14     12.0        False
147      0.00000  22161.01926 2023-10-12     37.0        False
121  29747.40747      0.00000 2023-10-20      2.0        False
102      0.00000  54309.34491        NaT      NaN        False
621      0.00000      0.00000 2023-05-02     85.0         True
733      0.00000  72713.06247 2024-03-21     31.0        False
543      0.00000  67370.55910 2023-04-05     67.0         True
482      0.00000      0.00000 2023-10-24     31.0        False
614  72351.37574      0.00000 2023-04-02     28.0        False
  AccountID  OverdraftCount
0  ACC30852               1
1  ACC45907               1
2  ACC49774               1
3  ACC50817               1
4  ACC57516               1
5  ACC57597               1
6  ACC57700               1
7  ACC61926               1
8  ACC69323               1
9  ACC77638               1
```

[21]:
```
##Calculate balance volatility using standard deviation or coefficient of␣
 ↪variation.

balance_std = df.groupby('AccountID')['AccountBalance'].std().reset_index()
balance_mean = df.groupby('AccountID')['AccountBalance'].mean().reset_index()

volatility = pd.merge(balance_std, balance_mean, on='AccountID')
```

```
volatility.columns = ['AccountID','BalanceStdDev','AvgBalance']

volatility['CV'] = volatility['BalanceStdDev'] / volatility['AvgBalance']

print(volatility)
```

```
    AccountID  BalanceStdDev     AvgBalance        CV
0    ACC10117            NaN   90780.256640       NaN
1    ACC10996   30390.798000   64046.568590  0.474511
2    ACC11062   37598.383500   62784.100737  0.598852
3    ACC11188   33781.833882   80558.926400  0.419343
4    ACC11285   42139.172552   95745.546255  0.440116
..        …            …           …       …
189  ACC97225   33590.603117   80994.270410  0.414728
190  ACC97411   34715.818903   61783.633875  0.561893
191  ACC99117   28817.571181   80478.450622  0.358078
192  ACC99409   30169.327975   32962.941265  0.915250
193  ACC99549   30812.319953  102738.124638  0.299911

[194 rows x 4 columns]
```

[22]:
```
#Use IQR or z-score methods to detect anomalies.
Q1 = df['TransactionAmount'].quantile(0.25)
Q3 = df['TransactionAmount'].quantile(0.75)

IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

anomalies_iqr = df[(df['TransactionAmount'] < lower_bound) |
                   (df['TransactionAmount'] > upper_bound)]

print(anomalies_iqr)
```

```
     TransactionID CustomerID AccountID AccountType TransactionType  \
97              67   CUST6391  ACC26940     Current         Payment
129            161   CUST4373  ACC41829     Current         Payment
701            191   CUST6837  ACC46655     Current         Deposit
53             178   CUST1189  ACC50439     Savings         Deposit
659             21   CUST1644  ACC77533      Credit         Payment
397             51   CUST1497  ACC88074     Current        Transfer
727            163   CUST1747  ACC92104     Savings         Deposit
242             86   CUST7855  ACC95164     Savings         Payment
579             50   CUST1749  ACC99117     Current         Payment
115            122   CUST8344  ACC99117      Credit      Withdrawal

              Product     Firm Region     Manager TransactionDate  \
```

```
97         Home Loan  Firm A  South  Manager 4      2023-03-16
129  Savings Account  Firm A  North  Manager 3      2024-03-22
701      Mutual Fund  Firm E   West  Manager 2      2024-06-16
53   Savings Account  Firm D   East  Manager 1      2024-06-16
659    Personal Loan  Firm D  South  Manager 4      2023-12-24
397      Mutual Fund  Firm B   East  Manager 3      2024-05-03
727  Savings Account  Firm A   West  Manager 1      2023-06-08
242        Home Loan  Firm C  North  Manager 3      2023-10-26
579    Personal Loan  Firm E   West  Manager 4      2023-11-29
115        Home Loan  Firm B   East  Manager 4      2024-01-29

     TransactionAmount  AccountBalance  RiskScore  CreditRating  TenureMonths  \
97         -25964.94612     94822.29734   0.639771           315           209
129        135972.34390     42302.00951  -0.368537           469           175
701        -29563.97803     49810.32678   0.517913           466           150
53         -60979.07348     57916.06101   0.160988           655            17
659        -30826.73980     68006.03943   0.488742           678            42
397        -45352.95439     96125.56424  -0.203847           668            36
727        147447.29510    101759.29850   0.099626           421            86
242        143067.18460     73627.79044   0.752617           582           226
579        -24819.84559    101124.98330   0.967253           628           202
115        129471.46560     73117.85560   0.186813           820           210

           Credit         Debit   PrevDate  GapDays  DormantFlag
97        0.00000  -25964.94612 2023-03-05     11.0        False
129       0.00000  135972.34390        NaT      NaN        False
701  -29563.97803       0.00000 2024-06-02     14.0        False
53   -60979.07348       0.00000 2024-05-03     44.0        False
659       0.00000  -30826.73980 2023-10-26     59.0        False
397       0.00000       0.00000 2024-04-25      8.0        False
727  147447.29510       0.00000        NaT      NaN        False
242       0.00000  143067.18460 2023-05-15    164.0         True
579       0.00000  -24819.84559 2023-11-18     11.0        False
115       0.00000  129471.46560 2023-11-29     61.0         True
```

[26]:
```python
mean = df['TransactionAmount'].mean()
std = df['TransactionAmount'].std()

df['ZScore'] = (df['TransactionAmount'] - mean) / std

anomalies_z = df[abs(df['ZScore']) > 3]

print(anomalies_z)
```

```
     TransactionID CustomerID AccountID AccountType TransactionType  \
53             178   CUST1189  ACC50439     Savings         Deposit
397             51   CUST1497  ACC88074     Current        Transfer
727            163   CUST1747  ACC92104     Savings         Deposit
```

```
242            86   CUST7855  ACC95164     Savings         Payment

             Product    Firm Region    Manager TransactionDate  …  \
53   Savings Account  Firm D   East  Manager 1      2024-06-16  …
397      Mutual Fund  Firm B   East  Manager 3      2024-05-03  …
727  Savings Account  Firm A   West  Manager 1      2023-06-08  …
242        Home Loan  Firm C  North  Manager 3      2023-10-26  …

     AccountBalance  RiskScore  CreditRating  TenureMonths        Credit  \
53      57916.06101   0.160988           655            17  -60979.07348
397     96125.56424  -0.203847           668            36       0.00000
727    101759.29850   0.099626           421            86  147447.29510
242     73627.79044   0.752617           582           226       0.00000

           Debit   PrevDate GapDays  DormantFlag    ZScore
53        0.0000 2024-05-03    44.0        False -3.854151
397       0.0000 2024-04-25     8.0        False -3.316740
727       0.0000        NaT     NaN        False  3.314026
242  143067.1846 2023-05-15   164.0         True  3.163385

[4 rows x 21 columns]
```

[30]:
```
Q1 = df['AccountBalance'].quantile(0.25)
Q3 = df['AccountBalance'].quantile(0.75)

IQR = Q3 - Q1

balance_anomalies = df[
    (df['AccountBalance'] < Q1 - 1.5*IQR) |
    (df['AccountBalance'] > Q3 + 1.5*IQR)
]
print (balance_anomalies)
```

```
     TransactionID CustomerID AccountID AccountType TransactionType  \
771            155   CUST1747  ACC11285     Savings      Withdrawal
629             12   CUST5545  ACC12182        Loan         Deposit
151            189   CUST7120  ACC18177        Loan         Payment
381            107   CUST8874  ACC21719     Current         Deposit
105             91   CUST8266  ACC28612     Savings         Payment
228             69   CUST3609  ACC29007        Loan         Payment
568              2   CUST7395  ACC30852     Current        Transfer
62              55   CUST8344  ACC33287      Credit      Withdrawal
121             41   CUST9535  ACC50817     Savings         Deposit
501            189   CUST4258  ACC53865        Loan         Payment
621            123   CUST9525  ACC57597     Savings        Transfer
543             31   CUST8461  ACC61926     Savings      Withdrawal
465             77   CUST1644  ACC62446        Loan        Transfer
```

```
            Product    Firm   Region    Manager TransactionDate  …  \
771        Home Loan  Firm C     East  Manager 2      2023-11-20  …
629        Home Loan  Firm A    South  Manager 4      2023-04-08  …
151      Mutual Fund  Firm B  Central  Manager 1      2024-06-22  …
381    Personal Loan  Firm D    South  Manager 3      2023-11-28  …
105      Mutual Fund  Firm D    North  Manager 1      2023-04-30  …
228      Mutual Fund  Firm A    North  Manager 3      2024-02-13  …
568    Personal Loan  Firm D     East  Manager 4      2023-09-12  …
62       Credit Card  Firm A     West  Manager 4      2023-06-06  …
121      Mutual Fund  Firm A     East  Manager 4      2023-10-22  …
501        Home Loan  Firm D     West  Manager 3      2023-10-06  …
621      Credit Card  Firm C  Central  Manager 4      2023-07-26  …
543      Credit Card  Firm A    South  Manager 2      2023-06-11  …
465  Savings Account  Firm A     West  Manager 3      2023-05-07  …

     AccountBalance  RiskScore  CreditRating  TenureMonths       Credit  \
771   162335.777300   0.622845           661           100      0.00000
629   159827.662600   0.522347           534            70  63230.96125
151   166805.957700   0.810727           848           208      0.00000
381   162192.896300   0.420651           822           237  50035.62595
105   164902.939700   0.913824           531            90      0.00000
228   159321.430000   0.591294           799           119      0.00000
568   -13018.400900   0.280604           320             8      0.00000
62    183836.933800   0.397574           710             7      0.00000
121   -13812.693060   0.199034           624           167  29747.40747
501   161037.819100   0.240570           348           199      0.00000
621   -12919.394960   0.254919           403            69      0.00000
543    -9976.162775   0.173516           475           170      0.00000
465   160065.347000  -0.207058           733           209      0.00000

            Debit   PrevDate GapDays  DormantFlag    ZScore
771   58554.13642 2023-10-24    27.0        False  0.256822
629       0.00000        NaT     NaN        False  0.417667
151   47390.87113        NaT     NaN        False -0.127104
381       0.00000 2023-05-02   210.0         True -0.036146
105   21116.34879 2023-04-30     0.0        False -1.030734
228   49643.03412 2023-10-06   130.0         True -0.049648
568       0.00000 2023-07-17    57.0        False -0.129017
62    41495.61915 2023-05-02    35.0        False -0.329853
121       0.00000 2023-10-20     2.0        False -0.733896
501  101272.54880        NaT     NaN        False  1.725989
621       0.00000 2023-05-02    85.0         True -0.475644
543   67370.55910 2023-04-05    67.0         True  0.560035
465       0.00000 2023-03-26    42.0        False -1.695313

[13 rows x 21 columns]
```

[32]:
```
#Highlight customers with irregular or suspicious transaction behavior.
import pandas as pd

mean = df['TransactionAmount'].mean()
std = df['TransactionAmount'].std()

df['ZScore'] = (df['TransactionAmount'] - mean) / std

suspicious_transactions = df[abs(df['ZScore']) > 3]

print (suspicious_transactions)
```

```
     TransactionID CustomerID AccountID AccountType TransactionType  \
53             178   CUST1189  ACC50439     Savings         Deposit
397             51   CUST1497  ACC88074     Current        Transfer
727            163   CUST1747  ACC92104     Savings         Deposit
242             86   CUST7855  ACC95164     Savings         Payment

              Product    Firm Region    Manager TransactionDate  …  \
53   Savings Account  Firm D   East  Manager 1      2024-06-16  …
397      Mutual Fund  Firm B   East  Manager 3      2024-05-03  …
727  Savings Account  Firm A   West  Manager 1      2023-06-08  …
242        Home Loan  Firm C  North  Manager 3      2023-10-26  …

     AccountBalance  RiskScore  CreditRating  TenureMonths        Credit  \
53      57916.06101   0.160988           655            17  -60979.07348
397     96125.56424  -0.203847           668            36       0.00000
727    101759.29850   0.099626           421            86  147447.29510
242     73627.79044   0.752617           582           226       0.00000

           Debit   PrevDate GapDays  DormantFlag    ZScore
53        0.0000 2024-05-03    44.0        False -3.854151
397       0.0000 2024-04-25     8.0        False -3.316740
727       0.0000        NaT     NaN        False  3.314026
242  143067.1846 2023-05-15   164.0         True  3.163385

[4 rows x 21 columns]
```

[37]:
```
withdrawals = df[df['TransactionType'] == 'Withdrawal']

withdrawal_counts = withdrawals.groupby('CustomerID').size().
 ↪reset_index(name='WithdrawalCount')

frequent_withdrawals = withdrawal_counts[withdrawal_counts['WithdrawalCount'] >␣
 ↪20]

print (frequent_withdrawals)
```

```
Empty DataFrame
Columns: [CustomerID, WithdrawalCount]
Index: []
```

[39]:
```
overdraft_accounts = df[df['AccountBalance'] < 0]

overdraft_customers = overdraft_accounts['CustomerID'].unique()

print (overdraft_customers )
```

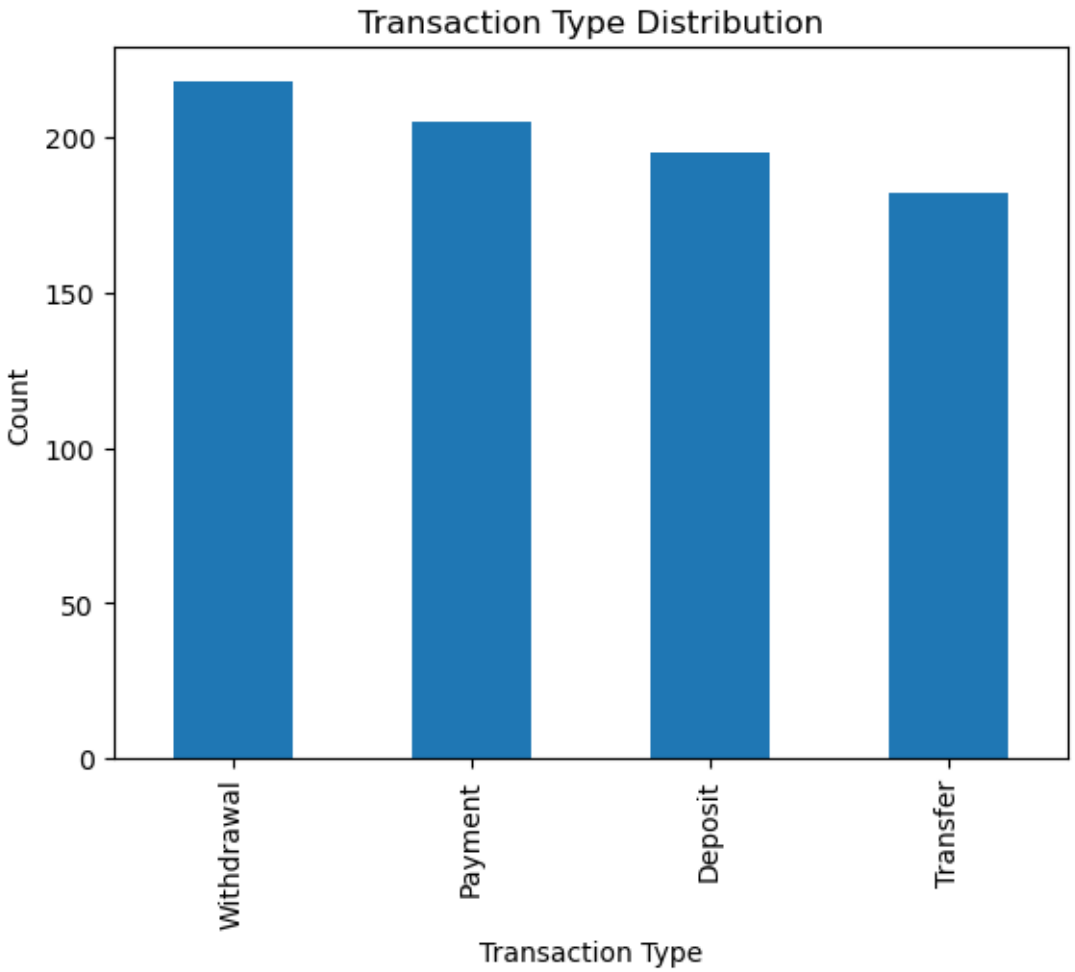
```
['CUST7395' 'CUST2915' 'CUST6028' 'CUST9535' 'CUST4769' 'CUST9525'
 'CUST4373' 'CUST8461' 'CUST6947' 'CUST8318']
```

[41]:
```
##Task 5: Visualisation
#Conduct extensive exploratory data analysis with attractive visualizations for␣
 ↪your
#findings
print(df.shape)
print(df.info())
print(df.describe())
```

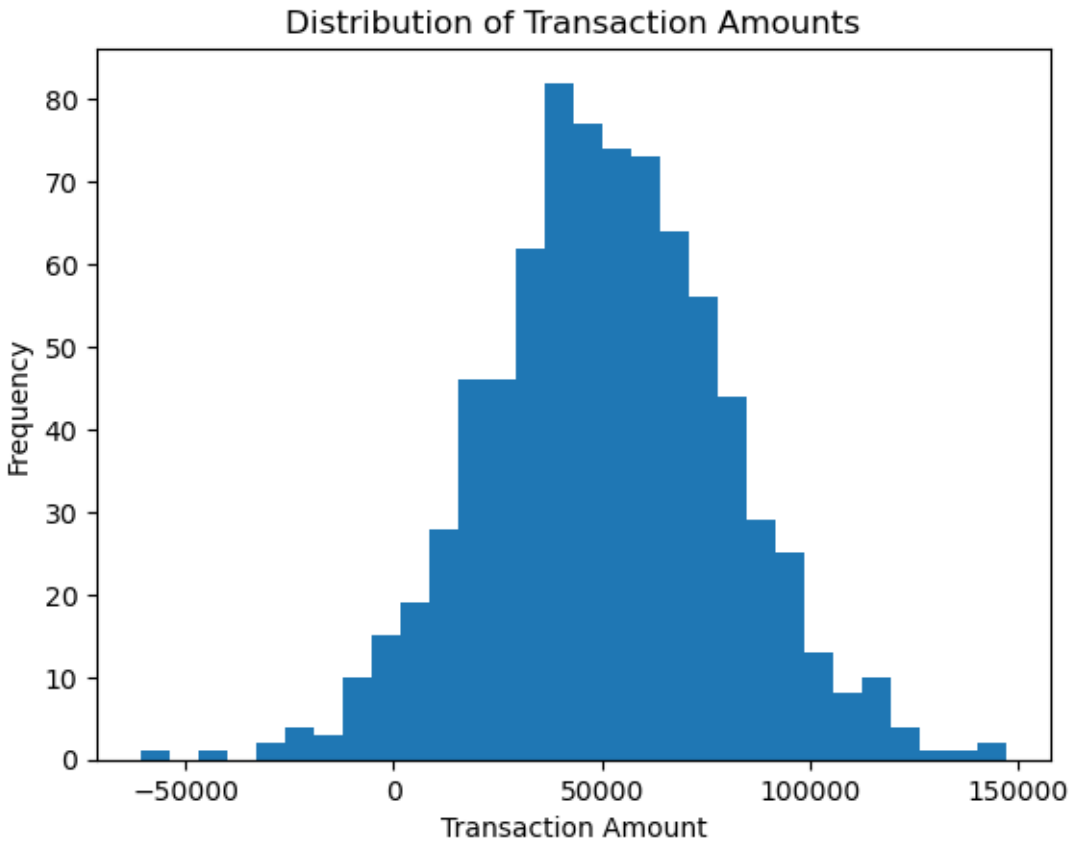
```
(800, 21)
<class 'pandas.core.frame.DataFrame'>
Index: 800 entries, 666 to 721
Data columns (total 21 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   TransactionID      800 non-null    int64
 1   CustomerID         800 non-null    object
 2   AccountID          800 non-null    object
 3   AccountType        800 non-null    object
 4   TransactionType    800 non-null    object
 5   Product            800 non-null    object
 6   Firm               800 non-null    object
 7   Region             800 non-null    object
 8   Manager            800 non-null    object
 9   TransactionDate    800 non-null    datetime64[ns]
 10  TransactionAmount  800 non-null    float64
 11  AccountBalance     800 non-null    float64
 12  RiskScore          800 non-null    float64
 13  CreditRating       800 non-null    int64
 14  TenureMonths       800 non-null    int64
 15  Credit             800 non-null    float64
 16  Debit              800 non-null    float64
 17  PrevDate           606 non-null    datetime64[ns]
 18  GapDays            606 non-null    float64
 19  DormantFlag        800 non-null    bool
 20  ZScore             800 non-null    float64
```

```
dtypes: bool(1), datetime64[ns](2), float64(7), int64(3), object(8)
memory usage: 132.0+ KB
None
       TransactionID                TransactionDate  TransactionAmount  \
count      800.00000                            800         800.000000
mean       101.96125  2023-09-27 13:06:35.999999744       51086.624542
min          1.00000            2023-01-06 00:00:00      -60979.073480
25%         49.00000            2023-05-15 00:00:00       32589.042120
50%        104.00000            2023-09-24 00:00:00       50307.224210
75%        155.25000            2024-01-29 00:00:00       70179.479290
max        199.00000            2024-12-03 00:00:00      147447.295100
std         58.27147                            NaN       29076.621451

       AccountBalance   RiskScore  CreditRating  TenureMonths         Credit  \
count      800.000000  800.000000    800.000000    800.000000     800.000000
mean     74528.005025    0.473350    576.303750    123.566250   13080.916582
min     -13812.693060   -0.368537    304.000000      6.000000  -60979.073480
25%      53547.221090    0.305702    433.000000     67.000000       0.000000
50%      73736.442995    0.461838    573.000000    123.000000       0.000000
75%      95584.413473    0.634402    722.000000    181.000000       0.000000
max     183836.933800    1.345638    849.000000    239.000000  147447.295100
std      32608.264579    0.245104    160.816133     66.426755   27140.557528

              Debit                        PrevDate     GapDays        ZScore
count     800.000000                             606  606.000000  8.000000e+02
mean    26622.444223  2023-08-05 03:21:58.811881216   99.415842 -3.819167e-16
min    -30826.739800            2023-01-06 00:00:00    0.000000 -3.854151e+00
25%         0.000000            2023-04-14 00:00:00   30.000000 -6.361668e-01
50%      4060.574895            2023-07-19 00:00:00   69.000000 -2.680505e-02
75%     52391.844735            2023-11-04 18:00:00  142.500000  6.566394e-01
max    143067.184600            2024-12-01 00:00:00  520.000000  3.314026e+00
std     32733.380965                            NaN   94.784725  1.000000e+00
```

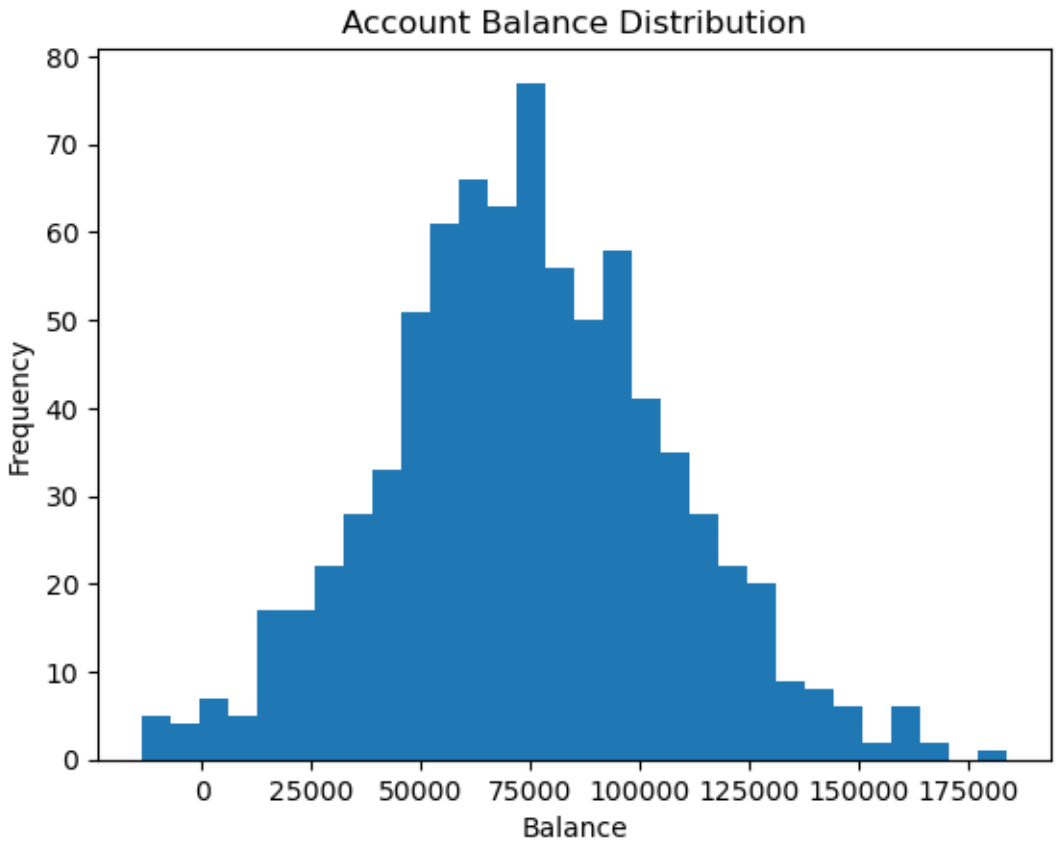
[42]:
```python
import matplotlib.pyplot as plt

df['TransactionType'].value_counts().plot(kind='bar')

plt.title("Transaction Type Distribution")
plt.xlabel("Transaction Type")
plt.ylabel("Count")

plt.show()
```



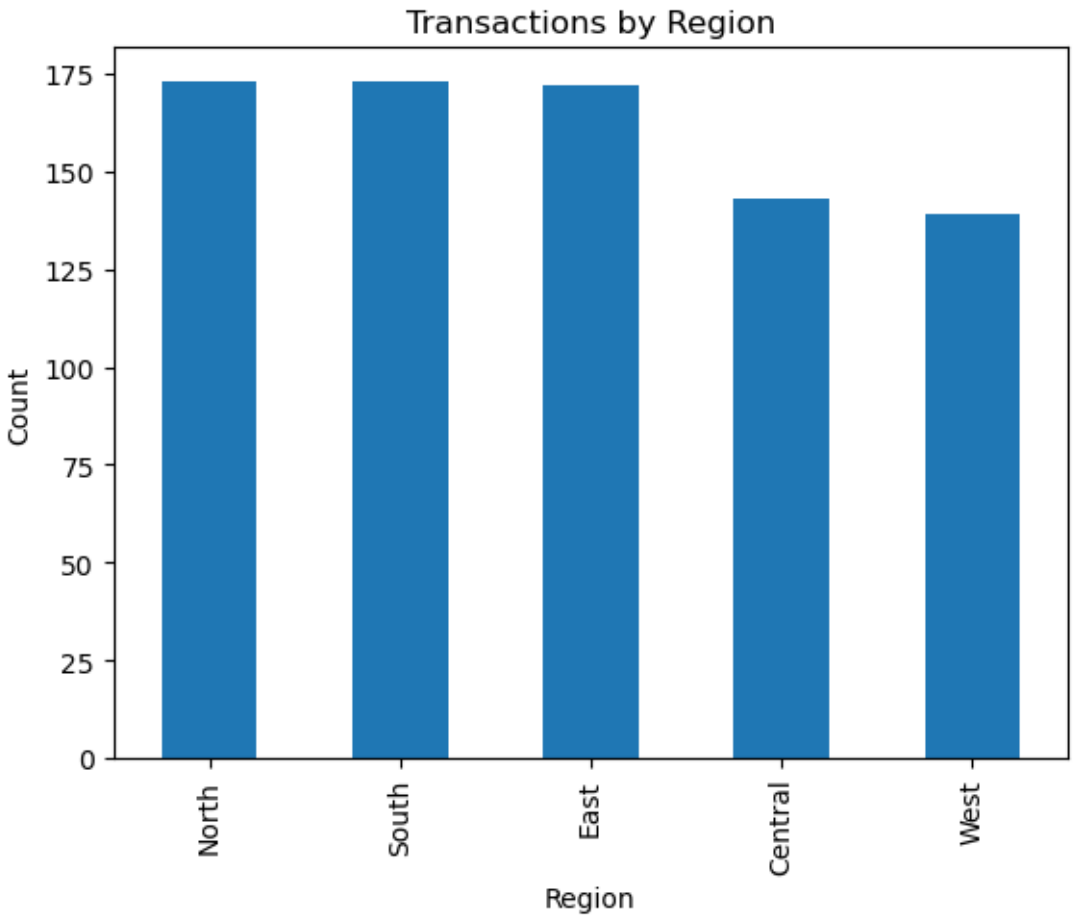
[43]:
```
df['TransactionAmount'].plot(kind='hist', bins=30)

plt.title("Distribution of Transaction Amounts")
plt.xlabel("Transaction Amount")

plt.show()
```



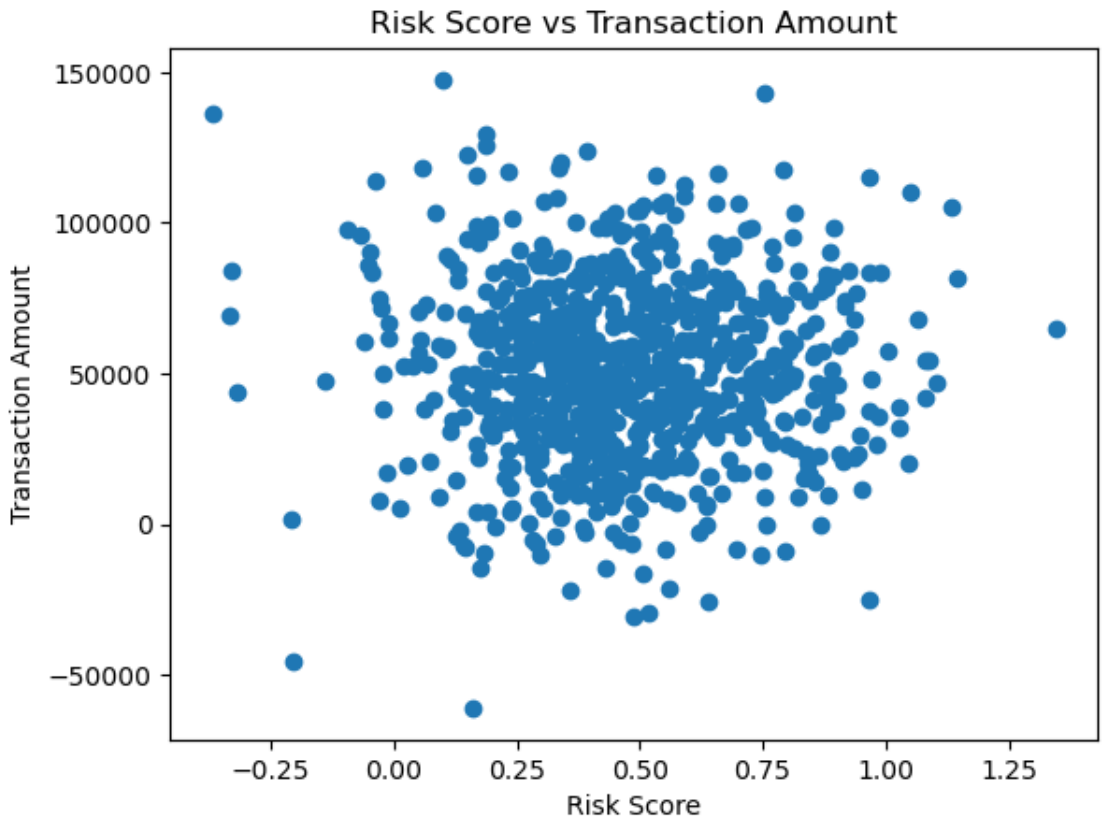
[44]:
```
df['AccountBalance'].plot(kind='hist', bins=30)

plt.title("Account Balance Distribution")
plt.xlabel("Balance")

plt.show()
```



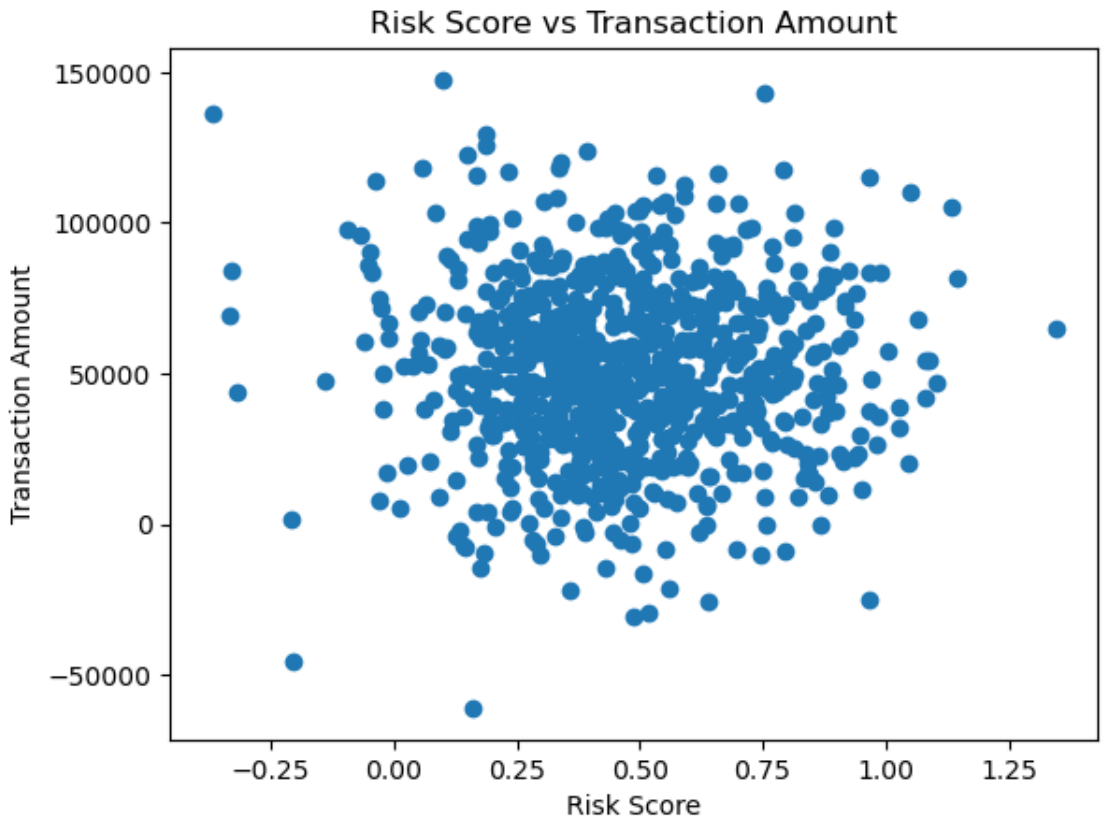
[45]:
```
df['Region'].value_counts().plot(kind='bar')

plt.title("Transactions by Region")
plt.xlabel("Region")
plt.ylabel("Count")

plt.show()
```



[46]:
```
plt.scatter(df['RiskScore'], df['TransactionAmount'])

plt.title("Risk Score vs Transaction Amount")
plt.xlabel("Risk Score")
plt.ylabel("Transaction Amount")

plt.show()
```



[47]:
```
plt.scatter(df['RiskScore'], df['TransactionAmount'])

plt.title("Risk Score vs Transaction Amount")
plt.xlabel("Risk Score")
plt.ylabel("Transaction Amount")

plt.show()
```



[51]:
```
##Task 6: Hypothesis Testing

#Test whether high-volume transaction accounts have statistically higher␣
 ↪average
#balances than low-volume accounts.
transaction_counts = df.groupby('AccountID').size().
 ↪reset_index(name='TransactionCount')
print (transaction_counts)
```

```
    AccountID  TransactionCount
0    ACC10117                 1
1    ACC10996                 4
2    ACC11062                 4
3    ACC11188                 3
4    ACC11285                 6
..        …                   …
189  ACC97225                 5
190  ACC97411                 6
191  ACC99117                 6
192  ACC99409                 2
193  ACC99549                 5
```

```
[194 rows x 2 columns]
```

[52]:
```
avg_balance = df.groupby('AccountID')['AccountBalance'].mean().
  ↪reset_index(name='AvgBalance')
print (avg_balance)
```

```
    AccountID     AvgBalance
0    ACC10117   90780.256640
1    ACC10996   64046.568590
2    ACC11062   62784.100737
3    ACC11188   80558.926400
4    ACC11285   95745.546255
..        …            …
189  ACC97225   80994.270410
190  ACC97411   61783.633875
191  ACC99117   80478.450622
192  ACC99409   32962.941265
193  ACC99549  102738.124638

[194 rows x 2 columns]
```

[54]:
```
account_data = pd.merge(transaction_counts, avg_balance, on='AccountID')
print (account_data)
```

```
    AccountID  TransactionCount     AvgBalance
0    ACC10117                 1   90780.256640
1    ACC10996                 4   64046.568590
2    ACC11062                 4   62784.100737
3    ACC11188                 3   80558.926400
4    ACC11285                 6   95745.546255
..        …                …            …
189  ACC97225                 5   80994.270410
190  ACC97411                 6   61783.633875
191  ACC99117                 6   80478.450622
192  ACC99409                 2   32962.941265
193  ACC99549                 5  102738.124638

[194 rows x 3 columns]
```

[56]:
```
high_volume = account_data[account_data['TransactionCount'] > 50]['AvgBalance']
low_volume = account_data[account_data['TransactionCount'] < 20]['AvgBalance']
print (high_volume)
print (low_volume)
```

```
Series([], Name: AvgBalance, dtype: float64)
0       90780.256640
1       64046.568590
```

```
2        62784.100737
3        80558.926400
4        95745.546255
            …
189      80994.270410
190      61783.633875
191      80478.450622
192      32962.941265
193     102738.124638
Name: AvgBalance, Length: 194, dtype: float64
```

[58]:
```python
##Conduct hypothesis testing based on segmentation.
import pandas as pd
from scipy.stats import ttest_ind

transaction_counts = df.groupby('AccountID').size().
 ↪reset_index(name='TransactionCount')
median_count = transaction_counts['TransactionCount'].median()

transaction_counts['Segment'] = transaction_counts['TransactionCount'].apply(
    lambda x: 'HighActivity' if x > median_count else 'LowActivity'
)

avg_balance = df.groupby('AccountID')['AccountBalance'].mean().
 ↪reset_index(name='AvgBalance')

account_data = pd.merge(transaction_counts, avg_balance, on='AccountID')

high_activity =␣
 ↪account_data[account_data['Segment']=='HighActivity']['AvgBalance']
low_activity =␣
 ↪account_data[account_data['Segment']=='LowActivity']['AvgBalance']

t_stat, p_value = ttest_ind(high_activity, low_activity, equal_var=False)

print("T-statistic:", t_stat)
print("P-value:", p_value)
```

```
T-statistic: -0.12571364439977387
P-value: 0.900090151280787
```

[ ]: